Quantum-Resilient Authenticated Communication for Connected Autonomous Vehicles

Review 2: the complete system and its evaluation

Harini R Ananyalakshmi V K Adhithya K Ratish S A

Guide: Dr. Vijayalakshmi M

Department of Information Technology, College of Engineering Guindy, Anna University

The problem

- V2X safety messages are signed with **ECDSA** and keyed with **ECDH** (IEEE 1609.2).
- **Shor's algorithm breaks both.** Traffic recorded today can be decrypted later.
- Cars stay on the road 15+ years: a car sold now must be secure in 2040.
- NIST standardised the replacements in 2024: **ML-KEM** (FIPS 203) and **ML-DSA** (FIPS 204).

The catch: size

	ECDSA P-256	ML-DSA-65
public key	65 B	1,952 B
signature	≈70 B	3,309 B
certificate	229 B	5,355 B

802.11p: 6 Mb/s, shared by every vehicle in range, beacons every 100 ms.

What we set out to do, and where we are

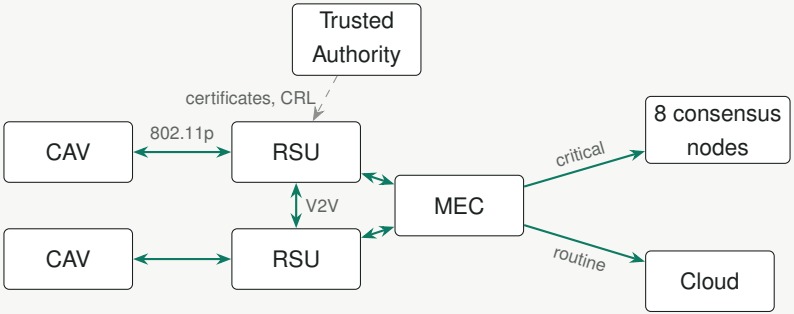
Objectives

1. Implement the base paper's framework end to end with ML-KEM and ML-DSA
2. Fix what prevents it from working
3. Measure compute, size and airtime vs a classical baseline
4. Evaluate on an 802.11p channel with mobility (ns-3)
5. Compare AGS-PBFT with PBFT
6. Attack it

Status

- ✓ Distributed system: 19 processes over TCP
- ✓ ns-3: 5 suites + caching $\times$ 50–200 vehicles $\times$ 3 seeds
- ✓ 21 attacks, 21 rejected
- ✓ 57 automated tests
- ✓ New: certificate caching (handshake $-51\,\%$)
- ✓ Replay viewer for the simulation

Architecture



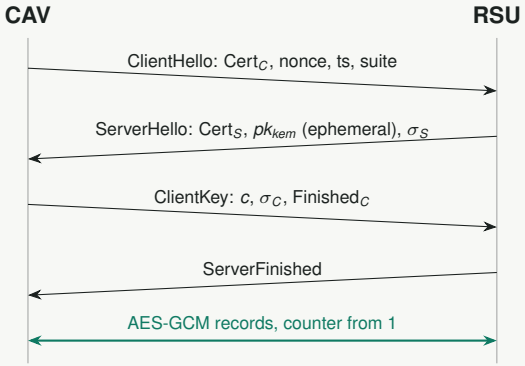
Every green link is a mutually authenticated ML-KEM + ML-DSA session carrying AES-256-GCM records. The MEC sends the **alert first**, then submits the event to consensus: safety never waits for the ledger.

The base paper cannot run as written

	In the paper	What we do
D1	Consensus messages “signed with Kyber”. **A KEM cannot sign.**	ML-DSA signatures
D2	Encapsulation defines k_i using k_i	standard Encaps / Decaps
D3	Session key $= H(k_i \parallel k_j')$ with $k_i = k_j'$	HKDF over secret + transcript hash
D4	TA generates private keys, yet claims “no key escrow”	entities make their own keys; proof of possession
D5	Hyperledger Fabric 2.2 uses Raft: cannot host PBFT	our own certified ledger

And four gaps in AGS-PBFT itself (D6–D9): $\sigma = 0$ at the first regroup, candidates that can never be promoted, undefined group sizing, an unreachable prepare quorum.

Handshake: four flights, both sides authenticated



Fixes to our Review 1 design:

- **F2: the client now signs.** Before, a copied (public) certificate was enough to impersonate a vehicle.
- F1: ClientHello nonce opens the transcript.
- F3: data counter starts at 1: no GCM nonce reuse.
- F5–F7: role in certificate, downgrade protection, uniform abort.

Forward secrecy: the RSU's KEM key is erased after each session.

New: certificate caching

Half the handshake is two 5.3 KB certificates. Peers usually hold them already:

- RSU certificates: **pre-distributed** with the CRL
- vehicle certificates: **edge directory** at the MEC after first contact

Send $H(\text{cert})$, 32 bytes, instead. Unknown reference $\rightarrow$ HelloRetry, then the full certificate.

Security: cached certificates are still verified every time (TA signature, CRL); the reference is inside both signatures; caching only after a completed handshake.

DECK	full	cached
ClientHello	5,446	162
ServerHello	10,303	4,981
ClientKey	4,960	4,960
ServerFinished	70	70
total (B)	**20,779**	**10,173**
radio frames	17	10

−51 %

AGS-PBFT

1. MEC $\rightarrow$ leader: signed REQUEST
2. PRE-PREPARE $\rightarrow$ consensus set (+ candidates)
3. PREPARE all-to-all; each node validates the event
4. RESPONSE $\rightarrow$ leader; $2f + 1$ match $\rightarrow$ signed block with commit certificate
5. Every node re-verifies the block, scores and membership

- Score $+1$ agree, -5 disagree
- Every k blocks: demote below $\mu - \sigma$, promote above $\mu + \sigma$
- View change when the leader is silent
- PBFT baseline with the same signatures

Implementation

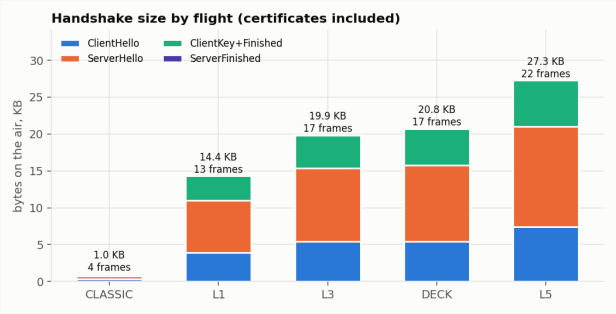
Runs three ways

1. **Distributed**: TA, cloud, MEC, 8 nodes, 3 RSUs, 5 vehicles as 19 processes; faults injected (Byzantine node, leader crash, revocation)
2. **ns-3**: IEEE 802.11p, Nakagami fading, Manhattan mobility, 9 RSUs; real message sizes and measured processing times
3. **Evaluation suite**: benchmarks, attacks, consensus experiments, figures

Stack

- Python, `cryptography` (OpenSSL 3.5: native ML-KEM / ML-DSA)
- `kyber-py`, `dilithium-py`: pure-Python comparison backend
- ns-3 3.44 (C++ scenario via Python bindings)
- 5 suites: Classical, L1, L3, DECK (1024/65), L5
- Classical runs through the **same protocol code**

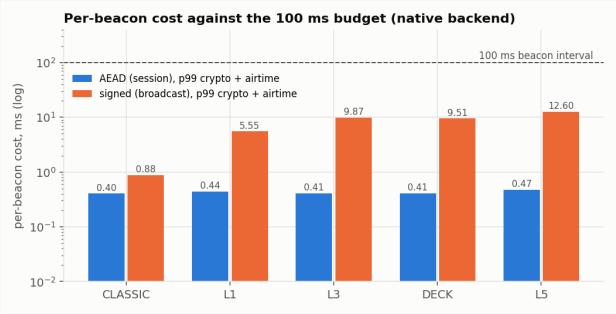
Result 1: compute is small, bytes are not



	Classical	DECK
bytes	1,043	20,779
frames	4	17
airtime (ms)	2.05	30.5
compute, native (ms)	1.16	4.77
compute, pure (ms)	–	209

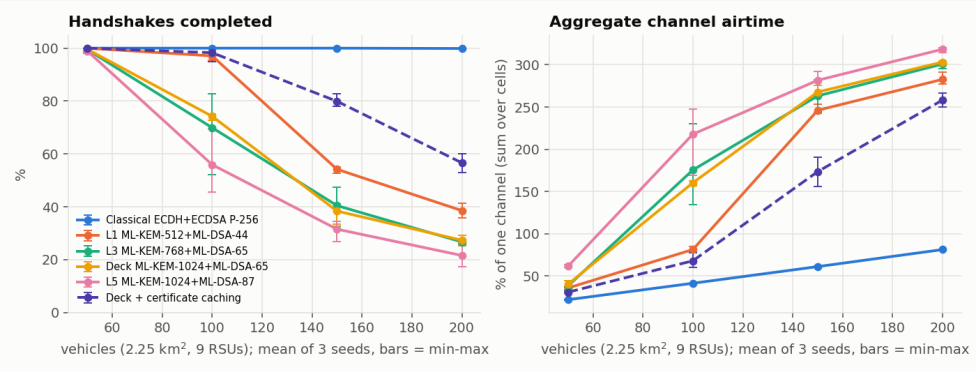
ML-DSA-65 signs in 1.28 ms natively vs 74.0 ms in pure Python: **implementation matters more than the algorithm**.

Result 2: per message, post-quantum fits



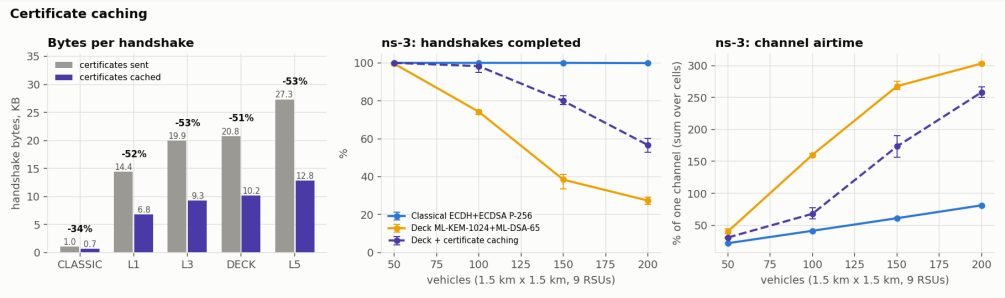
- In a session a beacon is a 135-byte AES-GCM record for **every** suite: 0.41 ms of a 100 ms budget.
- Signing every beacon fits natively (9.51 ms) but costs 3 frames per beacon.
- The base paper folds the one-off handshake into per-message latency.

Result 3: dense traffic breaks post-quantum handshakes



Classical: 100 % at 200 vehicles. DECK: 74 % at 100, 27 % at 200. Roughly in order of size (L3 ≈ DECK, within seed noise). **The limit is the channel, not the CPU.**

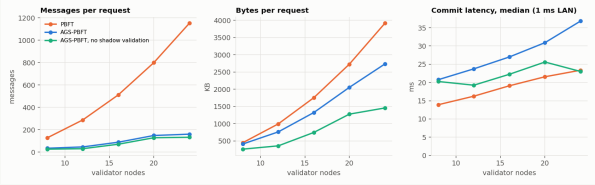
Result 4: caching and backoff recover much of it



At 100 vehicles DECK completes 74 % → **98 %** with caching; airtime 160 % → 68 %.

Without exponential backoff after failures: 7–11 % complete, alerts wait 380–430 ms behind handshakes.

Result 5: AGS-PBFT saves messages, not time



- **73–86 % fewer messages**; 18–25 % less CPU at light load, 65 % *more* under a burst
- Bytes only 8–30 % fewer: blocks carry $2f + 1$ ML-DSA signatures
- Latency **+41–58 %**: one more hop through the leader
- Beyond f Byzantine nodes progress stops (leader kept honest in these runs)
- Weakness: one outlier inflates σ and shields a silent node

Result 6: security

21 of 21 attacks rejected, each run against the real code:

- record tampering, replay, GCM nonce reuse
- ClientHello replay, stale timestamp
- client / RSU impersonation with a copied certificate
- rogue TA, revoked credential, vehicle posing as RSU
- MITM on the KEM key or ciphertext, downgrade, tampered Finished
- forward secrecy
- claiming another vehicle's cached certificate; revoked but cached
- forged / spoofed votes, tampered block, rewritten ledger

Client impersonation succeeds against the Review 1 design (no client signature).

Distributed run (live demo)

Secure sessions	48; 6 rejected (revoked vehicle)
Critical event $\rightarrow$ alert at vehicles	median 5.4 ms
Event $\rightarrow$ on the ledger	median 62 ms
Byzantine NODE_03	demoted at the first regroup; candidate promoted
Leader crash	view change, later events commit
Revocation	session closed, reconnection rejected
8 ledgers	consistent, verified from genesis (crashed leader's is a prefix)

`python scripts/run_network.py` then the replay viewer: `results/viewer.html`

Findings

1. **Per message, post-quantum V2X fits** the 100 ms budget at every security level, natively.
2. **Bandwidth is the binding constraint**: a PQ handshake is $\sim 20\times$ classical, and dense cells saturate.
3. **Certificate caching halves the handshake**; exponential backoff prevents congestion collapse.
4. **DECK = category 3**: L3 gives the same security with a smaller KEM.
5. **AGS-PBFT**: far fewer messages; slower; never on the safety path.

Limitations and next steps

Limitations

- Distributed system on one machine; radio only in ns-3
- ns-3 replays measured costs
- No WAVE module (ad-hoc MAC)
- Certificates linkable; no pseudonyms
- No formal verification

Next

- KEM-based authentication (KEMTLS-style) to shrink the handshake further
- Median/MAD regrouping for AGS-PBFT
- ProVerif / Tamarin model of the handshake
- Final report

Thank you

Questions